

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
(НИУ ИТМО)

Факультет программной инженерии и компьютерной техники

ОТЧЁТ
О ЛАБОРАТОРНОЙ РАБОТЕ № 4
Проектирование вычислительных систем
по теме:
Интерфейс I 2С и матричная клавиатура
Вариант 3

Выполнили: студенты группы Р3434

Р. Д. Баянов

Д. А. Кузнецов

Проверил:

преподаватель В. Ю. Пинкевич

Санкт-Петербург 2025

СОДЕРЖАНИЕ

Цель работы	3
Задание	4
Вариант	5
1 Описание организации программы и структуры драйверов, блок- схема прикладного алгоритма.....	6
2 Исходный код	8
ЗАКЛЮЧЕНИЕ.....	21

ЦЕЛЬ РАБОТЫ

1. Получить базовые знания об интерфейсе I2C и особенностях передачи данных по данному интерфейсу.
2. Получить базовые знания об устройстве и принципах работы контроллера интерфейса I2C в микроконтроллерах и получить навыки его программирования.

ЗАДАНИЕ

Разработать программу, которая использует интерфейс I2C для считывания нажатий кнопок клавиатуры стенда SDK-1.1.

Подсистема опроса клавиатуры должна удовлетворять следующим требованиям:

- реализуется защита отдребезга;
- нажатие кнопки фиксируется сразу после того, как было обнаружено, что кнопка нажата (с учетом защиты отдребезга), а не в момент отпускания кнопки; если необходимо, долгое нажатие может фиксироваться отдельно;
- кнопка, которая удерживается дольше, чем один цикл опроса, не считается повторно нажатой до тех пор, пока не будет отпущена (нет повторений);
- распознается и корректно обрабатывается множественное нажатие (при нажатии более чем одной кнопки считается, что ни одна кнопка не нажата, если это не противоречит требованиям к программе);
- всем кнопкам назначаются коды от 1 до 12 (порядок на усмотрение исполнителей).

Программа должна иметь два режима работы, переключение между которыми производится по нажатию кнопки на боковой панели стенда:

- режим тестирования клавиатуры;
- прикладной режим.

Уведомление о смене режима выводится в UART.

В режиме тестирования клавиатуры программа выводит в UART коды нажатых кнопок.

В прикладном режиме программа обрабатывает нажатия кнопок и выполняет действия в соответствии с вариантом задания.

ВАРИАНТ

Задания аналогичны вариантам лабораторной работы №3, за исключением того, что ввод символов должен выполняться не с клавиатуры через UART, а с помощью клавиатуры стенда. Выбор кнопок клавиатуры стенда, играющих роль кнопок клавиатуры компьютера должен выполняться по усмотрению исполнителей. В отчете необходимо привести описание функций кнопок в реализованной программе.

1 Описание организации программы и структуры драйверов, блок-схема прикладного алгоритма

Конфигурация I2C

I2C1 Mode and Configuration

Mode

I2C I2C

Configuration

Reset Configuration

GPIO Settings

NVIC Settings

DMA Settings

Parameter Settings

User Constants

Configure the below parameters :

Search (Ctrl+F)

Master Features

I2C Speed M... Standard Mode

I2C Clock S... 100000

Timing configuration

Coefficient o... 0

Analog Filter Enabled

Slave Features

Clock No Str... Disabled

Primary Add... 7-bit

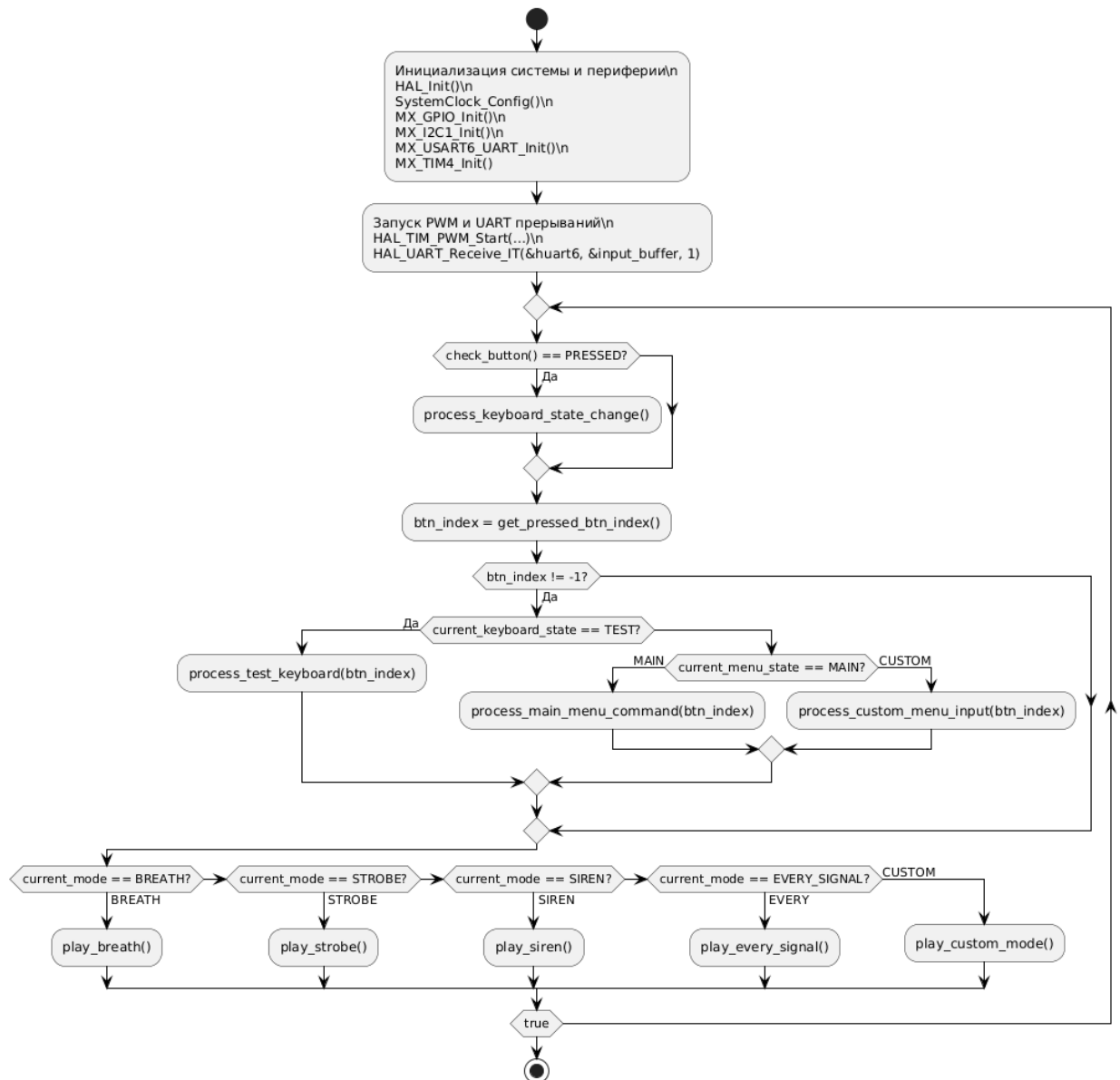
Dual Addres... Disabled

Primary slav... 0

General Call... Disabled

Блок-схема

Блок-схема программы main.c



2 Исходный код

```

/* USER CODE BEGIN Header */
/**
 * *****************************************************************************
 *
 * @file      : main.c
 * @brief     : Main program body
 *
 * *****************************************************************************
 *
 * @attention
 *
 * Copyright (c) 2025 STMicroelectronics.
 * All rights reserved.
 *
 * This software is licensed under terms that can be found in the
 LICENSE file
 * in the root directory of this software component.
 * If no LICENSE file comes with this software, it is provided AS-IS.
 *
 *
 *
 *
 *
 *
 */
/* USER CODE END Header */
/* Includes -----*/
#include "main.h"
#include "i2c.h"
#include "tim.h"
#include "usart.h"
#include "gpio.h"

/* Private includes -----*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----*/
/* USER CODE BEGIN PTD */

typedef enum {
    BREATH = 0,
    STROBE,
    SIREN,
    EVERY_SIGNAL,
    CUSTOM_MODE
} mode;

typedef enum {
    GREEN,
    YELLOW,
    RED
} indicator_color;

typedef struct {
    indicator_color color;
    uint32_t brightness;
    uint32_t duration;
} custom_mode_step;

```

```

typedef enum {
    MAIN,
    CUSTOM
} menu_state;

typedef enum {
    COLOR,
    BRIGHTNESS,
    DURATION
} custom_menu_state;

typedef enum {
    IN_PROGRESS,
    NOT_PRESSED,
    PRESSED,
} button_state;

typedef enum {
    TEST,
    NORMAL
} keyboard_state;

/* USER CODE END PTD */

/* Private define -----*/
/* USER CODE BEGIN PD */

#define MIN_BRIGHTNESS 0
#define MAX_BRIGHTNESS 100

#define MAX_CUSTOM_MODE_STEP_COUNT 5

#define KB_I2C_ADDRESS (0xE2)
#define KB_I2C_READ_ADDRESS ((KB_I2C_ADDRESS) | 1)
#define KB_I2C_WRITE_ADDRESS ((KB_I2C_ADDRESS) & ~1)
#define KB_INPUT_REG (0x0)
#define KB_OUTPUT_REG (0x1)
#define KB_CONFIG_REG (0x3)
#define KB_KEY_DEBOUNCE_TIME (200)

#define ANTI_BOUNCE_DELAY 40
#define PRESS_DELAY 100

/* USER CODE END PD */

/* Private macro -----*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables -----*/

/* USER CODE BEGIN PV */
uint32_t speed_percents = 50;

mode current_mode = BREATH;
char* mode_name[] = {
    "breath",
    "strobe",
    "siren",
    "every signal",

```

```

    "custom mode"
};

custom_mode_step custom_mode_steps[MAX_CUSTOM_MODE_STEP_COUNT];
uint32_t current_custom_mode_step_count = 0;
uint32_t current_custom_mode_step_index = 0;

uint8_t input_buffer;
uint8_t output_buffer[256];

menu_state current_menu_state = MAIN;

custom_menu_state current_custom_menu_state = COLOR;

keyboard_state current_keyboard_state = NORMAL;

uint32_t last_pressing_time = 0;
int last_pressed_btn_index = -1;
int stroka[4] = {0, 0, 0, 0};
int stolbec[3] = {0, 0, 0};

/* USER CODE END PV */

/* Private function prototypes -----*/
void SystemClock_Config(void);
/* USER CODE BEGIN PFP */
void wait(uint32_t delay);

void play_breath();
void play_strobe();
void play_siren();
void play_every_signal();
void play_custom_mode();

void process_main_menu_command(int);
void process_custom_menu_command(int);
void process_keyboard_state_change();
void process_test_keyboard(int btn_index);

int get_pressed_btn_index();

button_state check_button();
/* USER CODE END PFP */

/* Private user code -----*/
/* USER CODE BEGIN 0 */

/* USER CODE END 0 */

/**
 * @brief The application entry point.
 * @retval int
 */
int main(void)
{
    /* USER CODE BEGIN 1 */

    /* USER CODE END 1 */

    /* MCU Configuration-----*/

    /* Reset of all peripherals, Initializes the Flash interface and the
    SysTick. */

```

```

HAL_Init();

/* USER CODE BEGIN Init */

/* USER CODE END Init */

/* Configure the system clock */
SystemClock_Config();

/* USER CODE BEGIN SysInit */

/* USER CODE END SysInit */

/* Initialize all configured peripherals */
MX_GPIO_Init();
MX_I2C1_Init();
MX_USART6_UART_Init();
MX_TIM4_Init();
/* USER CODE BEGIN 2 */

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_3);
HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_4);

HAL_UART_Receive_IT(&huart6, &input_buffer, 1);
while (1)
{
    if (check_button() == PRESSED) {
        process_keyboard_state_change();
    }
    int btn_index = get_pressed_btn_index();
    if (btn_index != -1) {\
        if (current_keyboard_state == TEST) {
            process_test_keyboard(btn_index);
        } else {
            if (current_menu_state == MAIN) {
                process_main_menu_command(btn_index);
            } else {
                process_custom_menu_input(btn_index);
            }
        }
    }
    if (current_mode == BREATH) {
        play_breath();
    } else if (current_mode == STROBE) {
        play_strobe();
    } else if (current_mode == SIREN) {
        play_siren();
    } else if (current_mode == EVERY_SIGNAL) {
        play_every_signal();
    } else {
        play_custom_mode();
    }
}

/* USER CODE END WHILE */

/* USER CODE BEGIN 3 */
}
/* USER CODE END 3 */
}

/**

```

```

* @brief System Clock Configuration
* @retval None
*/
void SystemClock_Config(void)
{
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    /** Configure the main internal regulator output voltage
    */
    __HAL_RCC_PWR_CLK_ENABLE();
    __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE1);

    /** Initializes the RCC Oscillators according to the specified
    parameters
    * in the RCC_OscInitTypeDef structure.
    */
    RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSE;
    RCC_OscInitStruct.HSEState = RCC_HSE_ON;
    RCC_OscInitStruct.PLL.PLLState = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource = RCC_PLLSOURCE_HSE;
    RCC_OscInitStruct.PLL.PLLM = 15;
    RCC_OscInitStruct.PLL.PLLN = 216;
    RCC_OscInitStruct.PLL.PLLP = RCC_PLLP_DIV2;
    RCC_OscInitStruct.PLL.PLLQ = 4;
    if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
    {
        Error_Handler();
    }

    /** Activate the Over-Drive mode
    */
    if (HAL_PWREx_EnableOverDrive() != HAL_OK)
    {
        Error_Handler();
    }

    /** Initializes the CPU, AHB and APB buses clocks
    */
    RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYCLK
                                |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV4;
    RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV2;

    if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_5) !=
    HAL_OK)
    {
        Error_Handler();
    }
}

/* USER CODE BEGIN 4 */

void process_command_message(int command) {
    if (command == 1 || command == 2) {
        sprintf(output_buffer, "\r\nMode: %s\n",
mode_name[current_mode]);
    } else if (command == 3 || command == 4) {
        sprintf(output_buffer, "\r\nSpeed: %ld%%\n", speed_percents);
    } else if (command == 5) {
        sprintf(output_buffer, "\r\nMode: %s\nSpeed: %ld%%\n",
mode_name[current_mode], speed_percents);
    } else if (command == 12) {

```

```

        sprintf(output_buffer, "\r\nCustom menu mode is open\nChosen
step: RED, 50%, 100ms\n");
    }

    if (command == 1 || command == 2 || command == 3 || command == 4 ||
command == 5 || command == 12) {
        HAL_UART_Transmit(&huart6, output_buffer, strlen(output_buffer),
100);
    }
}

void process_main_menu_command(int command) {
    if (command == 1) {
        if (current_mode == CUSTOM_MODE) {
            current_mode = BREATH;
        } else {
            current_mode++;
        }
    } else if (command == 2) {
        if (current_mode == BREATH) {
            current_mode = CUSTOM_MODE;
        } else {
            current_mode--;
        }
    } else if (command == 3) {
        if (speed_percents <= 90) {
            speed_percents += 10;
        }
    } else if (command == 4) {
        if (speed_percents >= 10) {
            speed_percents -= 10;
        }
    } else if (command == 12) {
        current_menu_state = CUSTOM;
    }

    process_command_message(command);
}

void process_custom_menu_input(int command) {
    static _Bool is_first_input = 1;
    static indicator_color chosen_color = RED;
    static uint32_t chosen_brightness = 50;
    static uint32_t chosen_duration = 100;

    // Обработка ESC - выход из режима настройки
    if (command == 12) { // ESC
        current_menu_state = MAIN;
        is_first_input = 1;

        chosen_color = RED;
        chosen_brightness = 50;
        chosen_duration = 100;

        sprintf(output_buffer, "Main menu is open\n");
        HAL_UART_Transmit(&huart6, output_buffer,
strlen(output_buffer), 100);
        return;
    }

    sprintf(output_buffer, "Chosen step: %s, %lu%, %lums\n",
(chosen_color == RED) ? "RED" : (chosen_color == YELLOW) ?
"YELLOW" : "GREEN",
chosen_brightness,
chosen_duration);
}

```

```

HAL_UART_Transmit(&huart6, output_buffer, strlen(output_buffer),
100);

// Обработка Enter - парсинг введенной строки
if (command == 3) {
    if (current_custom_menu_state == COLOR) {
        current_custom_menu_state = BRIGHTNESS;
    } else if (current_custom_menu_state == BRIGHTNESS) {
        current_custom_menu_state = DURATION;
    } else {
        current_custom_menu_state = COLOR;
    }

    if (is_first_input) {
        current_custom_mode_step_count = 0;
        is_first_input = 0;
    }

    if (current_custom_mode_step_count <
MAX_CUSTOM_MODE_STEP_COUNT) {
        custom_mode_steps[current_custom_mode_step_count].color =
chosen_color;

        custom_mode_steps[current_custom_mode_step_count].brightness =
chosen_brightness;

        custom_mode_steps[current_custom_mode_step_count].duration =
chosen_duration;

        sprintf(output_buffer, "\nStep %d added: %s,
%lu%%, %lums\n",
current_custom_mode_step_count + 1,
(chosen_color == RED) ? "RED" :
(chosen_color == YELLOW) ? "YELLOW" : "GREEN",
chosen_brightness,
chosen_duration);
        HAL_UART_Transmit(&huart6, output_buffer,
strlen(output_buffer), 100);

        current_custom_mode_step_count++;

        if (current_custom_mode_step_count ==
MAX_CUSTOM_MODE_STEP_COUNT) {
            sprintf(output_buffer, "\r\nMaximum steps
reached (%d). Press ESC to finish.\n", MAX_CUSTOM_MODE_STEP_COUNT);
            HAL_UART_Transmit(&huart6, output_buffer,
strlen(output_buffer), 100);
            current_menu_state = MAIN;
            is_first_input = 1;
        } else {
            sprintf(output_buffer, "\r\nEnter next step
or press ESC to finish:\n");
            HAL_UART_Transmit(&huart6, output_buffer,
strlen(output_buffer), 100);
        }
    } else {
        sprintf(output_buffer, "\r\nMaximum steps reached
(%d). Press ESC to finish.\n", MAX_CUSTOM_MODE_STEP_COUNT);
        HAL_UART_Transmit(&huart6, output_buffer,
strlen(output_buffer), 100);
        current_menu_state = MAIN;
        is_first_input = 1;
    }

    chosen_color = RED;
    chosen_brightness = 50;
}

```

```

        chosen_duration = 100;
    }
}

if (command == 1) {
    if (current_custom_menu_state == COLOR) {
        if (chosen_color == RED) {
            chosen_color = GREEN;
        } else if (chosen_color == GREEN) {
            chosen_color = YELLOW;
        } else {
            chosen_color = RED;
        }
        sprintf(output_buffer, "Chosen color: %s\n",
(chosen_color == RED) ? "RED" : (chosen_color == YELLOW) ? "YELLOW" :
"GREEN");
    } else if (current_custom_menu_state == BRIGHTNESS) {
        if (chosen_brightness != 0) {
            chosen_brightness -= 10;
        }
        sprintf(output_buffer, "Chosen brightness: %d\n",
chosen_brightness);
    } else {
        if (chosen_duration != 0) {
            chosen_duration -= 100;
        }
        sprintf(output_buffer, "Chosen duration: %d\n",
chosen_duration);
    }
    HAL_UART_Transmit(&huart6, output_buffer, strlen(output_buffer),
100);
}

if (command == 2) {
    if (current_custom_menu_state == COLOR) {
        if (chosen_color == RED) {
            chosen_color = YELLOW;
        } else if (chosen_color == GREEN) {
            chosen_color = RED;
        } else {
            chosen_color = GREEN;
        }
        sprintf(output_buffer, "Chosen color: %s\n",
(chosen_color == RED) ? "RED" : (chosen_color == YELLOW) ? "YELLOW" :
"GREEN");
    } else if (current_custom_menu_state == BRIGHTNESS) {
        if (chosen_brightness != 100) {
            chosen_brightness += 10;
        }
        sprintf(output_buffer, "Chosen brightness: %d\n",
chosen_brightness);
    } else {
        if (chosen_duration != 1000) {
            chosen_duration += 100;
        }
        sprintf(output_buffer, "Chosen duration: %d\n",
chosen_duration);
    }
    HAL_UART_Transmit(&huart6, output_buffer,
strlen(output_buffer), 100);
}
}

void process_keyboard_state_change() {
    if (current_keyboard_state == NORMAL) {
        current_keyboard_state = TEST;
    }
}

```

```

        sprintf(output_buffer, "keyboard test mode activated\n");
    } else {
        current_keyboard_state = NORMAL;
        sprintf(output_buffer, "keyboard normal mode activated\n");
    }
    HAL_UART_Transmit(&huart6, output_buffer, strlen(output_buffer),
100);
}

void process_test_keyboard(int btn_index) {
    sprintf(output_buffer, "Pressed button %d\n", btn_index);
    HAL_UART_Transmit(&huart6, output_buffer, strlen(output_buffer),
100);
}

void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if (huart == &huart6) {
        if (current_menu_state == MAIN) {
            process_main_menu_command(input_buffer);
        } else {
            process_custom_menu_input(input_buffer);
        }
        HAL_UART_Receive_IT(&huart6, &input_buffer, 1);
    }
}

void wait(uint32_t delay) {
    if (delay == 0) return;
    uint32_t start_tick = HAL_GetTick();
    while (HAL_GetTick() - start_tick < delay) {
        int btn_index = get_pressed_btn_index();
        if (btn_index != -1) {
            process_main_menu_command(btn_index);
            return;
        }
    }
}

button_state check_button() {
    static _Bool was_pressed = 0;
    static uint32_t start_time = 0;

    if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_15) == GPIO_PIN_RESET) { /*
Кнопка на текущий момент нажата */
        if (!was_pressed) {
            was_pressed = 1;
            start_time = HAL_GetTick();
        }

        return IN_PROGRESS;
    } else { /* Кнопка на текущий момент не нажата */
        if (!was_pressed) {
            return NOT_PRESSED;
        }
        was_pressed = 0;

        uint32_t current_delay = HAL_GetTick() - start_time;
        if (current_delay >= PRESS_DELAY) {
            return PRESSED;
        }
    }
}

void play_breath() {
    static uint32_t brightness = 0;
    static uint32_t max_delay = 110;

```

```

static _Bool going_up = 1;

if (speed_percents == 0) {
    return;
}

htim4.Instance->CCR2 = brightness;
htim4.Instance->CCR3 = 0;
htim4.Instance->CCR4 = 0;

uint32_t actual_delay = max_delay * (110 - speed_percents) / 100;
wait(actual_delay);

if (going_up) {
    brightness += 10;
    if (brightness >= MAX_BRIGHTNESS) {
        brightness = MAX_BRIGHTNESS;
        going_up = 0;
    }
} else {
    brightness -= 10;
    if (brightness <= MIN_BRIGHTNESS) {
        brightness = MIN_BRIGHTNESS;
        going_up = 1;
    }
}
}

void play_strobe() {
    static uint32_t max_delay = 1000;

    if (speed_percents == 0) {
        return;
    }

    htim4.Instance->CCR3 = 0;
    htim4.Instance->CCR4 = 0;
    htim4.Instance->CCR2 = 100;

    // ?справлено: инвертированная формула скорости
    uint32_t actual_delay = max_delay * (110 - speed_percents) / 100;
    wait(actual_delay);

    htim4.Instance->CCR2 = 0;
    wait(actual_delay);
}

void play_siren() {
    static uint32_t brightness = 0;
    static uint32_t max_delay = 110;
    static _Bool going_up = 1;

    if (speed_percents == 0) {
        return;
    }

    htim4.Instance->CCR2 = brightness;
    htim4.Instance->CCR3 = 100 - brightness;
    htim4.Instance->CCR4 = 0;

    // ?справлено: инвертированная формула скорости
    uint32_t actual_delay = max_delay * (110 - speed_percents) / 100;
    wait(actual_delay);

    if (going_up) {
        brightness += 10;
    }
}

```

```

        if (brightness >= MAX_BRIGHTNESS) {
            brightness = MAX_BRIGHTNESS;
            going_up = 0;
        }
    } else {
        brightness -= 10;
        if (brightness <= MIN_BRIGHTNESS) {
            brightness = MIN_BRIGHTNESS;
            going_up = 1;
        }
    }
}

void play_every_signal() {
    static uint16_t pin = GPIO_PIN_13;
    static uint32_t max_delay = 1000;

    if (speed_percents == 0) {
        return;
    }

    if (pin == GPIO_PIN_13) {
        htim4.Instance->CCR2 = 100;
        htim4.Instance->CCR3 = 0;
        htim4.Instance->CCR4 = 0;
        pin = GPIO_PIN_14;
    } else if (pin == GPIO_PIN_14) {
        htim4.Instance->CCR2 = 0;
        htim4.Instance->CCR3 = 100;
        htim4.Instance->CCR4 = 0;
        pin = GPIO_PIN_15;
    } else {
        htim4.Instance->CCR2 = 0;
        htim4.Instance->CCR3 = 0;
        htim4.Instance->CCR4 = 100;
        pin = GPIO_PIN_13;
    }

    // ?справлено: инвертированная формула скорости
    uint32_t actual_delay = max_delay * (110 - speed_percents) / 100;
    wait(actual_delay);
}

void play_custom_mode() {
    if (current_menu_state == CUSTOM) {
        return;
    }

    if (current_custom_mode_step_count == 0) {
        return;
    }

    custom_mode_step current_custom_mode_step =
        custom_mode_steps[current_custom_mode_step_index];

    if (current_custom_mode_step.color == RED) {
        htim4.Instance->CCR2 = 0;
        htim4.Instance->CCR3 = 0;
        htim4.Instance->CCR4 = current_custom_mode_step.brightness;
    } else if (current_custom_mode_step.color == YELLOW) {
        htim4.Instance->CCR2 = 0;
        htim4.Instance->CCR3 = current_custom_mode_step.brightness;
        htim4.Instance->CCR4 = 0;
    } else {
        htim4.Instance->CCR2 = current_custom_mode_step.brightness;
        htim4.Instance->CCR3 = 0;
    }
}

```

```

        htim4.Instance->CCR4 = 0;
    }

    wait(current_custom_mode_step.duration);

    current_custom_mode_step_index++;
    if (current_custom_mode_step_index >=
current_custom_mode_step_count) {
        current_custom_mode_step_index = 0;
    }
}

int get_pressed_btn_index(){
    const uint32_t t = HAL_GetTick();
    if (t - last_pressing_time < KB_KEY_DEBOUNCE_TIME) return -1;
    int index = -1;
    uint8_t reg_buffer = ~0;
    uint8_t tmp = 0;
    int i = 0;
    HAL_I2C_Mem_Write(&hi2c1, KB_I2C_WRITE_ADDRESS, KB_OUTPUT_REG, 1,
&tmp, 1, KB_KEY_DEBOUNCE_TIME);
    for (int row = 0; row < 4; row++) {
        uint8_t buf = ~((uint8_t) (1 << row));
        HAL_I2C_Mem_Write(&hi2c1, KB_I2C_WRITE_ADDRESS, KB_CONFIG_REG, 1,
&buf, 1, KB_KEY_DEBOUNCE_TIME);
        HAL_Delay(10);
        HAL_I2C_Mem_Read(&hi2c1, KB_I2C_READ_ADDRESS, KB_INPUT_REG, 1,
&reg_buffer, 1, KB_KEY_DEBOUNCE_TIME);
        switch(reg_buffer >> 4){
            case 6: index = (i == 0) ? row * 3 + 1 : -1; i++; stroka[row] +=
1; stolbec[0] += 1; break;
            case 5: index = (i == 0) ? row * 3 + 2 : -1; i++; stroka[row] +=
1; stolbec[1] += 1; break;
            case 3: index = (i == 0) ? row * 3 + 3 : -1; i++; stroka[row] +=
1; stolbec[2] += 1; break;
            case 0:
            case 1:
            case 2:
            case 4:
                index = -1;
        }
    }
    if (index != -1) last_pressing_time = t;
    if (index == last_pressed_btn_index){
        return -1;
    }

    last_pressed_btn_index = index;

    int sum_stroka = 0;
    int sum_stolbec = 0;

    for (int j=0; j <= 3; j++){
        sum_stroka += stroka[j];
        stroka[j] = 0;
    }

    for (int j=0; j <= 2; j++){
        sum_stolbec += stolbec[j];
        stolbec[j] = 0;
    }

    if (sum_stroka != 1){
        return -1;
    }
    if (sum_stolbec != 1){

```

```

    return -1;
}

return index;
}

/* USER CODE END 4 */

/**
 * @brief This function is executed in case of error occurrence.
 * @retval None
 */
void Error_Handler(void)
{
    /* USER CODE BEGIN Error_Handler_Debug */
    /* User can add his own implementation to report the HAL error return
    state */
    __disable_irq();
    while (1)
    {
    }
    /* USER CODE END Error_Handler_Debug */
}

#ifdef  USE_FULL_ASSERT
/**
 * @brief Reports the name of the source file and the source line
    number
    where the assert_param error has occurred.
 * @param file: pointer to the source file name
 * @param line: assert_param error line source number
 * @retval None
 */
void assert_failed(uint8_t *file, uint32_t line)
{
    /* USER CODE BEGIN 6 */
    /* User can add his own implementation to report the file name and
    line number,
    ex: printf("Wrong parameters value: file %s on line %d\r\n", file,
    line) */
    /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */

```

ЗАКЛЮЧЕНИЕ

В ходе работы удалось настроить и запрограммировать микроконтроллер STM32F427VIT6 для работы с интерфейсом I2C и матричной клавиатурой.